

软件工程化说明文档

一、引言

本美术作品识别讲解网站致力于为用户提供便捷的美术作品识别、解读、交流及教育拓展服务。为保障项目开发质量、效率与可维护性，严格遵循软件工程化方法开展工作，以下对采用的软件工程化手段进行说明。

二、软件工程化手段应用

2.1 需求管理

参考 GJB 5000B - 2021 要求，对项目需求进行系统化管理与跟踪。

需求收集与分析：通过用户调研（如美术爱好者访谈、美术馆需求沟通）、市场竞品分析等方式，收集作品识别与鉴定、动态交互讲解、评价留言、教育扩展等模块需求，梳理出用户对作品识别准确性、讲解多语言支持、评价互动性及教育资源丰富度等核心诉求，进行需求分析与优先级排序。

需求规格化：将需求转化为清晰、可衡量的规格，如“作品识别与鉴定模块需支持手机摄像头实时捕捉及本地图片上传，调用 Qwen - VL 模型识别，输出作品基本信息（名称、作者等）、创作技巧（构图、色彩等）、情感表达等内容，识别响应时间 ≤ 30 秒”，采用用户故事、用例规约等形式描述，确保开发团队准确理解。

需求变更管理：使用需求管理工具，建立变更流程。若因业务拓展（如新增小众艺术流派识别需求）或技术优化（如提升模型适配性）需变更需求，需提交变更申请，经团队成员（评估影响（对进度、质量的影响））后，批准并更新需求文档与相关开发计划，确保需求变更可追溯、可控。

2.2 配置管理

对软件项目中的配置项（源代码、文档、测试数据等）进行有效管理。

版本控制：采用 Git 作为版本控制系统，搭建远程代码仓库（如 GitHub、GitLab）。按 GitFlow 或 GitHub Flow 分支策略管理代码，分为主分支（main，对应生产环境）、开发分支（develop，集成各功能开发）、功能分支（feature/xxx，开发具体功能，如 feature/artist - search 用于教育扩展模块的名人搜索功能开发）、发布分支（release/xxx，准备发布版本），实现代码版本的清晰管理与回退。

2.3 质量保证

通过多种技术与方法确保软件质量。

代码审查：采用定期代码评审会议 + 线上代码审查工具（如 GitHub Pull Request 审查机制）。开发人员完成功能分支开发后，提交合并请求，由团队成员进行代码审查，检查代码是否符合编码规范（如 JavaScript 代码遵循 ESLint 规则，HTML/CSS 遵循代码格式化与语义化要求）、是否存在逻辑漏洞（如识别结果处理异常、评价数据存储安全问题）、是否具备可维护性（代码结构清晰、注释完整），提出改进建议，通过后合并到开发分支。

测试覆盖：

单元测试：使用 Jest（针对 JavaScript 代码）等自动化测试工具，对关键函数（如图片上传处理函数、识别结果解析函数、评价发布逻辑函数）编写

单元测试用例，验证函数输入输出、异常处理是否符合预期，确保单个功能模块的正确性，要求单元测试覆盖率 $\geq 70\%$ 。

集成测试：搭建测试环境，模拟前后端交互（如前端上传图片，后端调用模型识别、返回结果并展示），使用 Selenium 或 Cypress 进行 UI 集成测试，验证各模块（作品识别、讲解、评价、教育扩展）之间交互流程（如识别成功后能否正常进入讲解界面、评价能否同步展示）、数据传递（如识别结果数据能否完整用于讲解内容生成、评价关联作品信息是否正确）是否正确，确保系统整体功能的协调性。

2.4 过程改进

持续优化软件开发过程，提升开发效率与质量。

开发方法与实践：采用敏捷开发方法，以迭代开发模式推进项目。将项目划分为多个迭代周期（如 2 周 / 迭代），每个迭代明确开发目标（如完成作品识别模块的基础功能开发、教育扩展模块的博物馆搜索功能 MVP 开发），制定迭代计划，包含需求拆分、任务分配（开发、测试任务）、进度跟踪。结合持续集成（CI），每个功能开发完成、代码提交后，自动触发构建与测试，快速反馈开发质量，及时修复问题；通过持续交付（CD），当迭代开发完成、测试通过后，自动部署到测试环境或预生产环境，缩短交付周期。

度量与指标评估：定义并跟踪关键度量指标，如代码覆盖率（要求单元测试覆盖率 $\geq 70\%$ ，随着项目推进逐步提升）、缺陷密度（统计每千行代码缺陷数，要求 ≤ 5 个 / 千行）、迭代周期（控制在 2 - 3 周，评估开发效率）、需求完成率（统计每个迭代中需求的实际完成情况，分析需求拆分合理性与开发进度把控）。定期（如每个迭代结束后）召开回顾会议，基于这些指标分析开发过程存在的问题（如缺陷密度高可能是代码审查不严格、测试用例覆盖不足；迭代周期过长可能是任务拆分不合理、团队协作不畅），制定改进措施（如优化代码审查流程、细化任务拆分），持续改进开发过程。

2.5 文档管理

对项目文档（需求文档、设计文档、用户手册等）进行规范管理。

文档创建与维护：使用腾讯文档作为文档管理协作平台，按模块、阶段分类管理文档。需求文档详细记录各模块需求（如作品识别模块的功能需求、交互流程）；设计文档包含架构设计（前后端分离架构、各模块技术选型与交互逻辑，如前端通过 RESTful API 调用后端识别接口）、数据库设计（评价数据、教育资源数据的表结构，如评价表包含用户 ID、作品 ID、评价内容、发布时间等字段）、接口设计（前后端交互接口的请求参数、响应格式、调用示例）；用户手册面向普通用户，介绍网站功能使用方法（如如何拍照识别作品、如何发布评价、如何搜索博物馆）、操作步骤与常见问题解答。文档需随项目迭代及时更新，确保与实际功能、设计一致。

协作与共享：团队成员通过腾讯文档在线编辑、评论文档，实现知识共享与协作。设置文档权限（如开发人员可编辑架构设计文档，测试人员可补充测试用例文档，产品与运营人员可查看用户手册并提出优化建议），保证文档的安全性与协同性，方便新成员快速熟悉项目，也为后续项目维护、功能扩展提供依据。

2.6 项目管理

保障项目进度、资源与风险可控。

工具与方法：将项目需求拆解为任务（如“开发作品识别拍照功能”“设计

评价留言界面”），分配给对应开发人员，设置任务优先级（高、中、低）、起止时间、依赖关系（如“作品识别接口开发完成”是“讲解内容展示功能开发”的前置任务）。通过看板（Kanban）方法管理项目，在 JIRA 中跟踪任务状态（待办、进行中、已完成），直观展示项目进度。

计划与风险管理：制定项目计划，明确里程碑（如完成作品识别模块开发与测试、整站功能联调完成）与交付时间。定期召开项目例会，汇报任务进度、解决遇到的问题（如接口联调报错、第三方 API 调用失败）。进行风险管理，识别项目风险（如模型识别准确率不达标、第三方教育资源 API 限制调用），评估风险影响程度与发生概率，制定应对策略（如针对模型准确率问题，增加样本数据优化模型；针对 API 限制，寻找替代资源或协商提升调用权限），确保项目按时交付、符合质量要求。

2.7 团队协作

促进团队成员（开发、测试、产品、设计）之间的高效协作与沟通。

协作工具：使用微信进行日常沟通，实时交流开发进度、问题解决思路。

三、总结

本美术作品识别讲解网站通过全面应用需求管理、配置管理、质量保证、过程改进、文档管理、项目管理、团队协作等软件工程化手段，构建了高效、可控、高质量的开发流程。这些手段相互配合，从需求到交付全流程保障项目的顺利推进，提升团队协作效率与软件质量，助力打造功能完善、体验优质的美术作品识别讲解平台，为用户提供专业、便捷的艺术服务，也为项目的持续迭代与维护奠定坚实基础。